

AGNEL INSTITUTE OF TECHNOLOGY AND DESIGN

```
import numpy as np

# Sigmoid activation function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of the sigmoid function
def sigmoid_derivative(x):
    return x * (1 - x)

# Error backpropagation class
class BackPropagationNetwork:
    def __init__(self, input_size, hidden_size, output_size, learning_rate=0.25):
        self.learning_rate = learning_rate

        # Input to hidden layer weights (including biases)
        self.W_input_hidden = np.array([
            [0.6, -0.3],
            [-0.1, 0.4],
            [0.3, 0.5]
        ])

        # Hidden to output layer weights (including biases)
        self.W_hidden_output = np.array([
            [0.4],
            [0.1],
            [-0.2]
        ])

        # Forward pass
        def forward_pass(self, X):
            # Add bias to input layer
            X = np.append(X, 1) # Bias input

            # Input to hidden layer
            self.hidden_input = np.dot(X, self.W_input_hidden) # Net input to hidden layer
            self.hidden_output = sigmoid(self.hidden_input) # Output from hidden layer

            # Add bias to hidden layer output
            self.hidden_output = np.append(self.hidden_output, 1) # Bias hidden

            # Hidden to output layer
            self.final_input = np.dot(self.hidden_output, self.W_hidden_output) # Net input to output layer
            self.final_output = sigmoid(self.final_input) # Output from output layer
```

AGNEL INSTITUTE OF TECHNOLOGY AND DESIGN

```
# Print net input and output for each layer
print(f'Net input to hidden layer: {self.hidden_input}')
print(f'Output from hidden layer: {self.hidden_output[:-1]}')
print(f'Net input to output layer: {self.final_input}')
print(f'Output from output layer: {self.final_output}')

return self.final_output

# Backpropagation phase
def backpropagate(self, X, Y):
    # Forward pass
    output = self.forward_pass(X)

    # Output layer error
    output_error = Y - output
    delta_output = output_error * sigmoid_derivative(output)

    # Hidden layer error
    hidden_error = np.dot(self.W_hidden_output[:-1], delta_output)
    delta_hidden = hidden_error * sigmoid_derivative(self.hidden_output[:-1])

    # Print the errors and deltas
    print(f'Output layer error: {output_error}')
    print(f'Delta for output layer: {delta_output}')
    print(f'Hidden layer error: {hidden_error}')
    print(f'Delta for hidden layer: {delta_hidden}')

    # Update weights for hidden to output layer
    W_hidden_output_update = self.learning_rate * np.outer(self.hidden_output, delta_output)
    self.W_hidden_output += W_hidden_output_update

    # Add bias to input layer
    X = np.append(X, 1)

    # Update weights for input to hidden layer
    W_input_hidden_update = self.learning_rate * np.outer(X, delta_hidden)
    self.W_input_hidden += W_input_hidden_update

    # Print weight updates
    print("Updated weights for hidden to output layer:")
    print(W_hidden_output_update)
    print("Updated weights for input to hidden layer:")
    print(W_input_hidden_update)
    print("=" * 50) # To separate epochs
```

AGNEL INSTITUTE OF TECHNOLOGY AND DESIGN

```
# Train the network
def train(self, X, Y, epochs=10000):
for epoch in range(epochs):
    print(f"Epoch {epoch+1}")
    self.backpropagate(X, Y)

# Test the network
def predict(self, X):
return self.forward_pass(X)

# Initialize the network
input_size = 2
hidden_size = 2
output_size = 1
learning_rate = 0.25
bpn = BackPropagationNetwork(input_size, hidden_size, output_size, learning_rate)

X = np.array([0, 1]) # X1 = 0, X2 = 1
Y = np.array([1])    # Target output Y = 1

# Train the network with this single input
bpn.train(X, Y, epochs=5)
# Test the network with the same input
predicted_output = bpn.predict(X)
print(f"Final predicted output: {predicted_output}, Target: {Y}")
```

AGNEL INSTITUTE OF TECHNOLOGY AND DESIGN

Output:

```
Epoch 1
Net input to hidden layer: [0.2 0.9]
Output from hidden layer: [0.549834 0.7109495]
Net input to output layer: [0.09102855]
Output from output layer: [0.52274144]
Output layer error: [0.47725856]
Delta for output layer: [0.11906782]
Hidden layer error: [0.04762713 0.01190678]
Delta for hidden layer: [0.0117885 0.00244685]
Updated weights for hidden to output layer:
[[0.01636688]
 [0.0211628 ]
 [0.02976695]]
Updated weights for input to hidden layer:
[[0.      0.      ]
 [0.00294713 0.00061171]
 [0.00294713 0.00061171]]
=====
Epoch 2
Net input to hidden layer: [0.20589425 0.90122342]
Output from hidden layer: [0.55129249 0.71120085]
Net input to output layer: [0.14547798]
Output from output layer: [0.53630549]
Output layer error: [0.46369451]
Delta for output layer: [0.11531244]
Hidden layer error: [0.04801228 0.01397158]
Delta for hidden layer: [0.01187675 0.00286968]
Updated weights for hidden to output layer:
[[0.01589272]
 [0.02050258]
 [0.02882811]]
Updated weights for input to hidden layer:
[[0.      0.      ]
 [0.00296919 0.00071742]
 [0.00296919 0.00071742]]
=====
```

```
Epoch 3
Net input to hidden layer: [0.21183263 0.90265826]
Output from hidden layer: [0.55276101 0.71149547]
Net input to output layer: [0.19832559]
Output from output layer: [0.54941952]
Output layer error: [0.45058048]
Delta for output layer: [0.11154467]
Hidden layer error: [0.04821626 0.01580202]
Delta for hidden layer: [0.01191984 0.00324367]
Updated weights for hidden to output layer:
[[0.01541439]
 [0.01984088]
 [0.02788617]]
Updated weights for input to hidden layer:
[[0.      0.      ]
 [0.00297996 0.00081092]
 [0.00297996 0.00081092]]
=====
Epoch 4
Net input to hidden layer: [0.21779255 0.9042801 ]
Output from hidden layer: [0.55423393 0.71182827]
Net input to output layer: [0.24956207]
Output from output layer: [0.56206871]
Output layer error: [0.43793129]
Delta for output layer: [0.10779568]
Hidden layer error: [0.04825732 0.01740968]
Delta for hidden layer: [0.01192239 0.00357123]
Updated weights for hidden to output layer:
[[0.01493601]
 [0.019183 ]
 [0.02694892]]
Updated weights for input to hidden layer:
[[0.      0.      ]
 [0.0029806 0.00089281]
 [0.0029806 0.00089281]]
=====
```

```
=====
Epoch 5
Net input to hidden layer: [0.22375375 0.90606571]
Output from hidden layer: [0.55570622 0.71219441]
Net input to output layer: [0.29919129]
Output from output layer: [0.57424481]
Output layer error: [0.42575519]
Delta for output layer: [0.10409191]
Hidden layer error: [0.04815396 0.01880829]
Delta for hidden layer: [0.01188906 0.0038552 ]
Updated weights for hidden to output layer:
[[0.01446113]
 [0.01853342]
 [0.02602298]]
Updated weights for input to hidden layer:
[[0.      0.      ]
 [0.00297226 0.0009638 ]
 [0.00297226 0.0009638 ]]
=====
Net input to hidden layer: [0.22969827 0.90799332]
Output from hidden layer: [0.55717341 0.71258936]
Net input to output layer: [0.34722844]
Output from output layer: [0.58594532]
Final predicted output: [0.58594532], Target: [1]
```